

BigCommerce + WSPay integration

Developer guide — architecture, flow, and codebase orientation.

ASP.NET Core service that integrates BigCommerce storefronts with WSPay as the payment processor. Runs as a side service that BigCommerce checkouts call into (via injected JS) and that WSPay calls back into. Not a public API and not consumer-facing — its only HTTP clients are the BigCommerce checkout page, WSPay, and an internal admin UI.

Stack

- **ASP.NET Core** (`Client.Web`), recently moved to .NET 10
- **NHibernate** via FluentNHibernate (`Persistency.NHibernate`)
- **FluentMigrator** for schema migrations (`Persistency.Migrations`)
- **Autofac** for DI — modules: `Domain.Services.Module` , `Client.Web.Module` , `Persistency.NHibernate.Module`
- **WebStudio** internal framework — `IQueryExecutor` , `ICommandExecutor` , `UnitOfWork` , `Response<T>` , command/query handler pattern (NOT MediatR)
- **RestSharp** for outbound HTTP (BigC REST Management API, WSPay form API)
- **JWT** bearer auth for admin endpoints
- **SQL Server** backend
- **Vanilla JS** front-end glue injected into the BigC checkout DOM (`wwwroot/monri_custom_checkout.js`)

Project layout

- `Domain/Model/` — entities (Stores, Shops, Transactions, Users, Administrators)
- `Domain/Services/` — command/query handlers, business logic, external API clients
 - `Callback/` — `SaveCallback` , `OrderStatusUpdate` , `SaveTransaction`
 - `Checkout/` — `InitiateCheckout` , `GetIFrameValues`
 - `WSPayForm/` — outbound WSPay form-API client (`IFormWSPay`)
 - `WSPay/` — WSPay backoffice ops (void/refund), `WSPaySignatureGenerator`
 - `RestManagementApi/` — BigCommerce REST client
 - `Helpers/` — `WsPayFormHelper` , `TransactionsHelpers` , `ResponseHelper` , `ErrorMessages` , `CacheStore`
- `Domain/Projections/` — read-side query DTOs
- `Persistency/NHibernate/` — mappings + query handlers
- `Persistency/Migrations/` — FluentMigrator migrations
- `Client/Web/` — controllers, request/response DTOs, `Program.cs` , `appsettings.json` , `wwwroot/monri_custom_checkout.js`

Key entities

- **StoreConfiguration** — one per BigCommerce store. Holds `AccessToken`, `ApiPath`, `MerchantSiteUrl`, `StoreStatus` (Test/Production), `CheckoutType` (EmbeddedCheckout/Redirect), `PaymentMethodName`, `OrderConfirmationRedirectRoute`, `DefaultOrderStatus`.
- **Shops** — N rows per `StoreConfiguration`, each holding WSPay credentials (`ShopID`, `SecretKey`, `Language`, `IsTokenShopId`). `IsTokenShopId` is a WSPay quirk: WSPay represents "this merchant account is used for tokenized payments" as a separate shop record. Don't read it as "this row stores a token" — it means "this row is the tokenization-enabled WSPay shop."
- **Transaction** — created/updated on every WSPay callback. Keyed by (`ShoppingCartID`, `ShopID`). Holds `WSPayOrderId`, `Amount`, status flags, customer fields.
- **User + UserTokens** — saved-card records, keyed by (`customer email`, `StoreConfiguration`).

Configuration

`appsettings.json` holds infrastructure-level config:

- `DatabaseConfiguration` — SQL connection
- `MSSQLServerLoggerConfiguration` — log table sink
- `FormWSPayConfiguration` — WSPay form API base URLs + timeouts
- `WSPayPaymentGatewayConfiguration` — WSPay backoffice URLs + basic auth
- `BigCommerceConfiguration` — OAuth client id/secret + `AppUri` (the public hostname of *this* service)
- `JwtSettings` — admin auth

Per-store data lives in DB (`StoreConfiguration` + `Shops`).

Checkout flow

The customer reaches the BigC checkout. BigC serves the page; in it is `monri_custom_checkout.js`, served from this app's `wwwroot`.

1. JS calls `POST /checkout/check-merchant-integration` with `{ SiteUrl }` → returns `"EmbeddedCheckout"` or `"Redirect"` (defaults to `EmbeddedCheckout` if no store match).
2. JS calls `POST /checkout/fetch-payment-method-name` → label for the radio button (default: "Plaćanje karticama").
3. JS uses a `MutationObserver` to inject a payment-option radio + container into the BigC checkout DOM.
4. When the customer picks "Monri", the flow branches on integration type.

Embedded flow

- JS calls `POST /checkout/get-iframe-values` with `{ CheckoutID, SiteUrl }`.
- `GetIframeValuesHandler` looks up the store, fetches BigC checkout (`/checkouts/{id}`) + create-order response, then loops the store's `Shops` rows and produces one `GetIframeValuesResult` per shop — each with WSPay form URL, signature, customer/order fields, `IsTokenShopID` flag.
- JS separates token vs non-token results, builds a hidden form per result, submits into `wspay-iframe`. WSPay renders its card form inside the iframe. Saved-card dropdown lets the user pick from `UserTokens` for that customer email.

Redirect flow

- JS calls `POST /checkout/init` with `{ CheckoutID, SiteUrl }`.
- `InitiateCheckoutHandler` does the BigC lookup + create-order with retry (up to $15 \times 2s$ for the create-order response), then calls `IFormWSPay.CreateFormTransactionAsync` against the WSPay form API. Returns `PaymentFormUrl`.
- Handler caches `{ ShopID, Currency, SiteUrl }` keyed by BigC order id (`IWebAppCache` / `IMemoryCache` , 30-min default TTL).
- JS attaches the URL to a "Pay" button. Click navigates the browser to WSPay.

Callback flow

After payment, WSPay calls back to:

1. `POST /callback/savetransaction` → `SaveCallbackHandler` :
 - Upsert `Transaction` keyed by `(ShoppingCartID, ShopID)`.
 - If `(CustomerEmail, Token)` present, upsert `User` + `UserTokens` for the saved-card flow.
 - Compute status via `TransactionsHelpers.CheckForStatusChangeAction` → `WSPayTransactionStatus` (Completed / Voided / Refunded / PartiallyRefunded).
 - Call BigC's order-update endpoint. Payment method label is hardcoded `"WSPay by Monri"`.
2. `POST /callback/orderstatusupdate` → `OrderStatusUpdateHandler` :
 - Used for void/refund flows initiated from our backoffice. Talks to WSPay's backoffice API.

Post-payment redirect

After payment, WSPay redirects the customer to `{AppUri}/redirect/order-confirmation`.

`RedirectController` looks up the cached `{ ShopID, Currency, SiteUrl }` by order id and bounces the customer back to the configured `OrderConfirmationRedirectRoute` on the BigC store.

Admin surface

`AdministratorController` at `/administrator`, JWT-protected after `/login`:

- CRUD on `StoreConfiguration` and `Shops`
- Read-side queries: `GetStoreConfigurationsHandler`, `GetShopsHandler`, `GetMerchantOptionsHandler`
- Log search: `GetLogsHandler`

No SPA in this repo — the admin UI lives elsewhere and talks to these endpoints.

Caching

`WebAppCache` wraps `IMemoryCache`. Holds short-lived per-order context (`ShopID`, `Currency`, `SiteUrl`). 30-min TTL by default. Used by the post-payment redirect and `UpdateOrderHandler`.

Logging

Structured logs go to a SQL Server `Log` table via `MSSQLServerLoggerConfiguration`. Every external call logs request/response/error keyed by a request id. Event categories live in `Domain.Services.Logging.Events`.

Deployment

Single-host ASP.NET Core app. Static `wwwroot` serves the JS shipped to BigC. The app's public hostname (`AppUri`) is both what gets injected into BigC pages and what WSPay calls back to. Default `appsettings.json` points at `monribigcomm.wspay.info`.

Hot spots / WSPay coupling to be aware of

The codebase is WSPay-coupled throughout. The places listed below are the ones you'll touch first if you ever want to support a second gateway.

- `WSPayFormHelper.CreateIframeRequest` builds the WSPay-shaped iframe payload and hardcodes the form URLs (`formtest.wspay.biz/authorization.aspx` for test, `form.wspay.biz/authorization.aspx` for prod).
- `WSPayFormHelper.CreateIframeRequest:78` has a hardcoded per-merchant override: `shop?.ShopID == "SKOLSKA" ? "self" : "TOP"` for the iframe response target.
- `SaveCallbackHandler` hardcodes the payment method label `"WSPay by Monri"` when updating BigC orders.
- `InitiateCheckoutHandler` picks the WSPay form URL from `FormWSPayConfiguration.BaseUrlProd / BaseUrlTest` based on `StoreStatus`.
- `ProcessingTransactionCallbackRequest` + `SaveCallbackCommand` are WSPay-shaped DTOs (`WSPayOrderId`, `STAN`, `ApprovalCode`, `Signature`, `ECI`, etc).
- `monri_custom_checkout.js` is WSPay-coupled throughout — element IDs `wspay-iframe`, `wspayiframeshopID`, `wspayiframesignature`, etc. The file mixes the embedded and redirect branches based on the response of `check-merchant-integration`.
- `Shops.IsTokenShopId` semantics: WSPay-specific. See entity note above.

Things this doc intentionally doesn't cover

- The admin UI's UX (lives in a different repo).
- WSPay onboarding (merchant agreement, API credentialing).
- BigCommerce app install / OAuth handshake (handled by `AuthController`, but a separate concern from runtime checkout).